

1. A Task Is Not a Wish

"Improve this" can be a valid opening move in an interactive thread, but it is not yet a task contract. A task contract is the smallest statement that lets the agent distinguish success from a merely impressive intermediate state. It normally contains an object, an intended change, constraints that materially affect the solution, and a finish condition.

Consider: "Make the test suite faster." The object is visible, but the target and boundary are not. A stronger contract is: "Reduce the median CI test time for this package by at least 25% without reducing coverage or changing public behavior; measure the baseline and final result with the same command." The number is not magic. Its value is that it makes failure legible.

2. The Exit-Criterion Argument

1. Long-running agent work requires a rule for deciding whether to continue.
2. A vague objective does not determine that rule reliably.
3. A verifiable criterion can be checked against evidence produced during the run.
4. Therefore a long-running goal should, where possible, contain a verifiable completion criterion.

Dominik Kundel makes this point about Codex goal mode: the goal prompt functions not only as an initial instruction but as an exit criterion. Numerical targets often help because they create an observable comparison, but non-numerical criteria can work when the state is still testable: "all public endpoints preserve the documented response schema" or "the browser flow succeeds from a fresh account."

3. The Proxy Problem

A measurable target can be wrong. If the agent is asked only to lower test time, deleting expensive tests satisfies the number while defeating the purpose. A good contract therefore separates the metric from invariants. Write the desired movement and the things that must not be traded away.

A useful form is:

Goal: G improves.

Evidence: E measures G closely enough for this task.

Invariants: I1...In must remain true.

Boundary: B marks actions that need approval.

Stop: return when E meets the target and I remains verified, or when a stated blocker makes the target infeasible.

4. Specification versus Procedure

Over-specification can turn a capable agent into a brittle macro. If the user dictates every command before the repository is inspected, the procedure encodes assumptions that may already be false. Under-specification produces the opposite failure: Astra may infer a product decision the user considers essential.

The right question is not "How detailed should the prompt be?" It is "Which facts are preferences and which are discoverable?" A personal visual preference, legal boundary, compatibility requirement, or acceptable data-loss policy should be explicit. The location of a handler, the existing test command, or the project naming convention should usually be discovered from the project unless the user has reason to override it.

5. A Counterexample

Suppose the task is "make this landing page feel finished." There may be no single metric. The contract can still be verifiable by specifying reference material and review dimensions: match the supplied design system; preserve all existing content; remove layout overflow at 375, 768, and 1440 pixels; verify keyboard focus; and present before/after screenshots. The task remains partly judgmental, but the evidence surface is much sharper.

References: Dominik Kundel, "A guide to /goal"; OpenAI, Codex use cases; OpenAI, GPT-6 Astra model guidance.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 2: The Task Contract

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>